

Loss Curves, Optimizers, BatchNorm, Dropout, Early Stopping

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 3, Day 20, Self-study, Olympiad practice

8 October 2026

1. Where This Fits
2. Learning Outcomes
3. Introduction
4. Common Tricks
5. Common Mistakes
6. Tips
7. Common Practices
8. Olympiad Practice
9. Check Yourself
10. Limitations and What Comes Next
11. Summary
12. References

- ▶ **This week gave you the training run:** the four moves and autograd (Day 15); the losses, the loop with batches, validation inside it and the curve as a dashboard (Day 16); the dials, measured one by one: optimizers, schedules, BatchNorm, dropout, early stopping (Day 17); activations and the shape (Day 18); the data side (Day 19).
- ▶ **This deck is about the run itself:** how it is planned, read and finished under a clock, so that the number you hand in is the best one you could have had. Real tasks are decided here. The Polish 2025 noisy-labels task fixes the model and the epochs and scores only how you train; IOAI 2024's on-site tasks allow five minutes of CPU per task; the Polish 2025 ECG task paid 35 points and 97 points for the same tiny model, trained carelessly and carefully (Day 13).
- ▶ **How to use it:** one move per frame: the situation, then the action, with code where the action is code. Every number comes from running that code on the ECG beats of Days 16 and 17 (Polish AI Olympiad 2025, 2,000 training beats, 1,500 validation beats, five classes, balanced accuracy). Keep the routine and the decision table open on Friday.

Measured on the ECG beats, same dials every time

- ▶ A forgotten `model.eval()` scores the same network 0.934 instead of 0.971: five points lost to dropout and BatchNorm left in training mode.
- ▶ Inputs left unscaled: 0.659 instead of 0.944 for the same plain network and the same 60 epochs.
- ▶ Submitting the last epoch instead of the best: 0.934 instead of 0.944. Trusting one seed: five seeds of the recipe score 0.971 to 0.975, and their average answer scores 0.979.
- ▶ A training set with 15% wrong labels drops the recipe from 0.972 to 0.888. The per-row loss, the same loss function with `reduction="none"`, is what the Polish 2025 noisy-labels task is scored on.

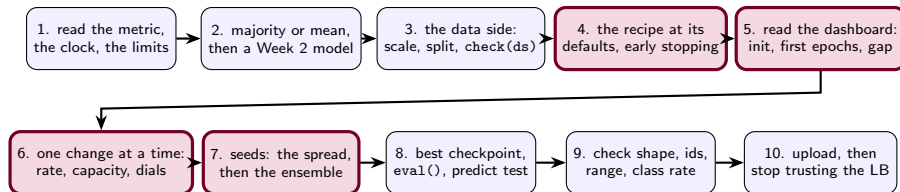
After this deck you can

run the recipe with the clock in view, read the dashboard every epoch and name the failure from its shape, apply the moves that win points and avoid the ones that lose them, and hand in a model whose hidden-test score is the one you saw on validation.

By the end of this self-study deck, you will be able to:

- ▶ **Plan** a training run under a clock: the metric, the baselines, the recipe at its defaults, the budget for changes, seeds and the submission.
- ▶ **Read** the dashboard every epoch (two losses, the judge's metric, the best epoch, the gap, the time) and name the failure from the shape of the curve.
- ▶ **Apply** the moves that win points: the rate sweep, the kept checkpoint with patience, class weights, seeds and their ensemble, the per-row loss on noisy labels, the sanity checks at initialisation.
- ▶ **Avoid** the moves that lose them: validating in training mode, submitting the last epoch, unscaled inputs, one seed, the wrong loss for the judge's metric, a leaked validation, no submission on the clock.
- ▶ **Judge** a change against the spread of seeds, change one thing at a time, and log every run.
- ▶ **Finish** with the submission routine for a network: best checkpoint, `eval()`, batches in order, the file checked.

Training Runs: **Introduction**



- Steps 1, 2, 9 and 10 are Day 13's routine; step 3 is Day 19. The purple steps are today's: they decide whether the model you hand in is the best the clock allowed.
- On the ECG beats the routine fits in minutes (the recipe trains in 1.7 s, five seeds in 9 s); on images each step costs more, so the order matters more.

The Dashboard: What to Print Every Epoch

Number	How	What it tells you
training loss	mean over the epoch's batches	whether the network can fit at all; $\ln k$ at init for k classes
validation loss	<code>eval()</code> , <code>no_grad</code> , whole set	the gap to the training loss: the overfitting alarm
the judge's metric	the same function the task scores	the number that chooses the checkpoint
best so far, best epoch	kept with <code>state_dict()</code>	whether the run is still improving
seconds per epoch	<code>time.perf_counter()</code>	how many epochs and seeds the clock allows

```
print(f"epoch {epoch:3d}  train {train_loss:.3f}  val {val_loss:.3f}  "  
      f"bal_acc {val_bal:.3f}  best {best:.3f}@{best_epoch}  {secs:.1f}s")
```

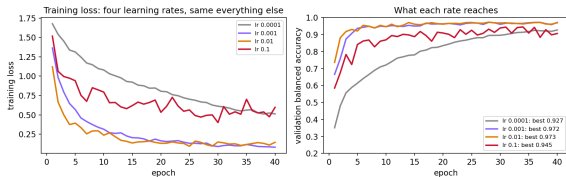
- The recipe on the ECG beats, epoch 1: training loss 1.363 against 1.683 before the first step ($\ln 5 = 1.609$; 1.620 with class weights). A first epoch that does not move the loss is the first thing the dashboard catches. Print the line; plot only when the run ends.

What the dashboard shows	Most likely cause	What to change
loss is nan or grows loss stuck at $\ln k$ from the start	rate too big rate wrong either way; inputs not scaled; labels the wrong dtype; a dead output activation	rate $\nabla \cdot 10$ rate $\times 10$ and $\nabla \cdot 10$; standardise; int64; last layer linear (Day 18)
falls, but slowly, still falling at the end	rate too small	rate $\times 10$ (0.0001 here: 0.927 and still climbing at epoch 40)
jumps up and down, never settles training falls, validation turns up	rate too big; batches too small overfitting	rate $\nabla \cdot 10$ (0.1 here: 0.945 best, 0.906 last) keep the best epoch; dropout, weight decay; stop earlier
both stay high	underfitting	wider, then deeper; more epochs; better columns
validation metric far below the validation loss suggests validation better than training the metric flat, the loss rising scores differ between runs by 0.01	eval() forgotten dropout on in training only; or a leak wrong answers getting confident seed noise	model.eval() before validating (0.934 against 0.971) normal with dropout; otherwise check the split nothing: keep the best metric epoch (Day 16) three seeds before believing a change; ensemble them
one class's recall far below the others	imbalance	class weights; threshold on OOF (Day 13)

Day 16's table with four new rows; every number in brackets is from the ECG runs of this deck.

Training Runs: **Common Tricks**

Trick: Sweep the Rate by Powers of Ten, Read the First Epochs



Source: the recipe network on the ECG beats, AdamW, four rates, 40 epochs each, same seed; scripts/day20.practice-figures.py.

```
for lr in [1e-4, 1e-3, 1e-2, 1e-1]:  
    torch.manual_seed(0); m = make_recipe()  
    opt = make_opt(m, lr)    # AdamW, wd 0.01  
    hist, best, _ = fit(m, opt, loss_fn, 10)
```

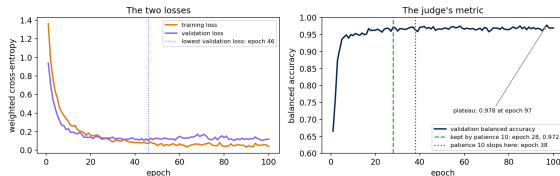
When it applies

Every new task, and again after a change of model, batch size or optimizer: the rate is the dial that matters most (Day 17; Goodfellow et al. 2016, section 11.4.1).

What to do

Four runs, same seed, ten epochs each. Here 0.0001 is still climbing at epoch 40 (0.927); 0.001 and 0.01 reach 0.972 and 0.973 by epoch 28; 0.1 jumps (0.945 best, 0.906 last). Keep the largest rate whose training loss falls smoothly.

Trick: Keep the Best Checkpoint, Stop With Patience



Source: the recipe on the ECG beats, AdamW 0.001, 100 epochs without stopping, the patience-10 decision drawn on top; `scripts/day20.practice-figures.py`.

```
if val_bal > best:
    best, best_epoch = val_bal, epoch
    state = copy.deepcopy(model.state_dict())
elif epoch - best_epoch >= patience:
    break
model.load_state_dict(state)
```

When it applies

Every run. The epoch count is never a dial you guess: the checkpoint is chosen afterwards, on the judge's metric (Day 16; the Tuning Playbook's retrospective checkpoint selection).

What to do

Patience 10 stops at epoch 38 and keeps epoch 28: 0.972. Running on to 100 finds 0.978 at epoch 97: 0.006 for 2.6 times the time, barely above the seed spread. Epochs cost a second here, so use patience 20 to 30; on images, 5 to 10. The validation *loss* bottoms at epoch 46 and rises while the metric stays flat (confident wrong answers): choose on the metric.

Trick: Weight the Loss When the Judge Reads Balanced Accuracy

When it applies

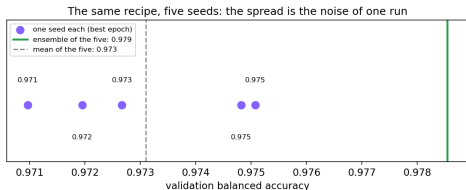
The metric treats every class alike (balanced accuracy, macro F1, recall of the rare class) while the rows do not: 1,400 normal beats against 150 of each disorder.

recipe, same seed	balanced accuracy	normal	afib	pac	pvc	st_elev
without class weights	0.967	1.000	0.972	0.932	0.970	0.960
with class weights 2,000/(5 · count)	0.972	0.982	0.972	0.921	0.985	1.000

```
counts = np.bincount(y_train) # [1400, 150, 150, 150, 150]
w = torch.tensor(counts.sum() / (5 * counts), dtype=torch.float32)
loss_fn = nn.CrossEntropyLoss(weight=w) # 0.286 and 2.667: 9.3 to 1
```

- ▶ Half a point here, where the network already separates the classes; on tables with a 2% class the same line is worth tens of points. It is the loss's version of Day 13's `class_weight="balanced"`.
- ▶ Print the per-class recall: balanced accuracy is their mean, and the lowest one says which class the next change is for.

Trick: Five Seeds, Then Average Their Softmax



Source: the recipe with early stopping, seeds 0 to 4, 9 s in total on the ECG beats; scripts/day20_practice_figures.py.

```
probs = []
for seed in range(5):
    torch.manual_seed(seed); m = make_recipe()
    _, best, _ = fit(m, opt_for(m), loss_fn,
                    100, patience=10)
    m.load_state_dict(best[2]); m.eval()
    with torch.no_grad():
        p = torch.softmax(m(X_test), 1)
        probs.append(p)
pred = torch.stack(probs).mean(0).argmax(1)
```

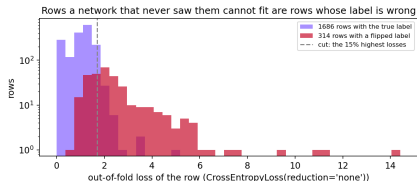
When it applies

Whenever one run fits in the remaining time five times over. Karpathy's recipe calls ensembles "a guaranteed way to gain 2% of accuracy on anything"; on this table the gain is half a point.

What to do

Train the same recipe with seeds 0 to 4: 0.972, 0.975, 0.975, 0.973, 0.971. Average the five softmax outputs and take the argmax: **0.979**, above every single seed. The spread (0.004) is also the noise every later change is compared against.

Trick: Let the Out-of-Fold Loss Find the Wrong Labels



Source: 314 of the 2,000 ECG training labels flipped at random, the recipe trained 12 epochs on each of 5 stratified folds, loss of every held-out row; scripts/day20-practice_figures.py.

```
row_loss = nn.CrossEntropyLoss(reduction="none")
cv = StratifiedKFold(5, shuffle=True,
                    random_state=0)
oof = torch.zeros(len(y))
for tr, va in cv.split(X, y):
    m = train_for(12, X[tr], y[tr]); m.eval()
    with torch.no_grad():
        oof[va] = row_loss(m(X[va]), y[va])
keep = oof <= oof.quantile(0.85) # top 15% out
```

When it applies

Labels made by tired annotators, by rules or by another model. The Polish 2025 noisy-labels task fixes the network and the epochs and scores only which rows you train on, from these per-row losses.

What to do

With 15% of labels wrong the recipe falls from 0.972 to 0.888. A network that never saw a row cannot fit a wrong label: median out-of-fold loss 2.08 for flipped rows, 1.12 for true ones. The 15% highest losses are 74% flipped rows; dropping them and retraining gives **0.912**. In-sample losses catch only half: a network fits the rows it has seen, wrong label and all.

Trick: Check the Loss at Init, Then Overfit One Batch

When it applies

Before the first real run of any new pipeline. “Neural net training fails silently” (Karpathy 2019): a wrong pairing or a dead output does not crash, it trains to a mediocre number.

```
m.eval()
with torch.no_grad():
    print(loss_fn(m(X_train), y_train).item())    # 1.683; ln 5 = 1.609
xb, yb = X_train[:64], y_train[:64]
opt = torch.optim.Adam(m.parameters(), lr=1e-3)
for step in range(300):                          # one batch, until ~0
    loss = loss_fn(m(xb), yb)
    opt.zero_grad(); loss.backward(); opt.step()
print(loss.item())    # 1.68, 0.076 at 50, 0.004 at 100, 0.0005 at 300
```

- **Loss at initialisation:** a fresh network guesses, so the cross-entropy sits near $\ln k$: 1.683 against 1.609 (1.620 with class weights). Far above it: unscaled inputs or an activation on the output (Day 18); exactly 0: a leak.
- **One batch to zero:** 64 rows, Adam 0.001, the loss must vanish in a few hundred steps. If it cannot memorise 64 rows, nothing downstream matters; a mismatched (x, y) pairing is the usual cause.

Trick: Start From the Week 2 Model and Beat It

Model on the ECG beats	balanced accuracy	time
majority (every beat normal)	0.200	0 s
logistic regression, balanced	0.554	0.1 s
random forest, 300 trees, balanced	0.931	1.0 s
plain 150-128-64-5 network, SGD, 60 epochs	0.944	1.1 s
the recipe with early stopping	0.972	1.7 s

When it applies

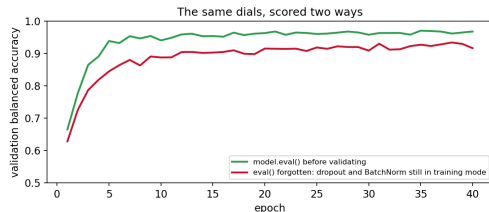
Every table and every signal that fits in a table. The network is the third model you train, never the first.

What to do

Hand in the forest before touching a network: it is the floor the network must beat and the file that saves the day if the loop breaks. Here the recipe wins by four points; on NEOAI 2025's table task gradient boosting was the whole solution, and on JOAI 2025 the sensor columns went into boosting models while the networks read the images. When the forest wins, submit the forest.

Training Runs: **Common Mistakes**

Mistake: Validating in Training Mode



Source: one recipe run on the ECG beats, the validation set scored after every epoch with and without `model.eval()`; `scripts/day20_practice_figures.py`.

```
def evaluate(model, X, y):  
    model.eval()           # dropout off  
    with torch.no_grad():  # no graph  
        logits = model(X)  
    model.train()          # back to training  
    return metric(y, logits.argmax(1))
```

The situation

Dropout keeps dropping units and BatchNorm keeps using the batch's own statistics while the validation set is scored. The same dials score 0.934 instead of 0.971; the gap averages 0.053 over 40 epochs and never closes.

- ▶ The quiet version: the checkpoint is chosen on a noisy, too-low number, and the test set is predicted in training mode as well.
- ▶ `eval()` and `no_grad()` belong inside `evaluate`, so no loop can forget them.

The situation

The loop ends, `model` holds the dials of the last epoch, and `model(X_test)` is called. Plain network, 60 epochs of SGD: best 0.944 at epoch 51, last 0.934. Recipe, 100 epochs without stopping: 0.978 at epoch 97, 0.970 at 100. Day 16's 1,000-image run: 0.885 at epoch 13, worse after.

```
hist, best, _ = fit(model, opt, loss_fn, 100, patience=10)
best_score, best_epoch, best_state = best
model.load_state_dict(best_state)      # the epoch the metric chose
torch.save(best_state, "best.pt")     # and keep it on disk
```

- ▶ The last epoch is a random point on the plateau, or past it. The kept epoch is the one the judge's metric chose; `load_state_dict` before predicting is the whole fix.
- ▶ Keep it on disk as soon as it improves: a crashed notebook then costs minutes, not the run.

The situation

The ECG voltages look harmless: between -1.13 and 2.21 . Their spreads per column run from 0.031 to 0.579 , twenty-fold apart, and the plain network trained on them reaches 0.659 instead of 0.944 . Day 18's reason: a unit's z is dominated by the wide columns and sits in the flat part of its activation.

```
mean, std = X_train.mean(0), X_train.std(0)          # training rows only
std[std < 1e-6] = 1.0
X_train, X_val = (X_train - mean) / std, (X_val - mean) / std
X_test = (X_test - mean) / std                      # the same mean and std
```

- ▶ It is the spread per column that matters, not the size of the numbers. Standardise every table and every signal; images get $/255$ and their mean and spread (Day 19).
- ▶ Trees (Day 10) do not care, which is why the forest ran fine on the raw values and the network did not. The forest's score on raw inputs is not a sign the scaling is optional.

The situation

A change moves the score from 0.972 to 0.975 and gets adopted. The same recipe with seeds 0 to 4 scores 0.972, 0.975, 0.975, 0.973, 0.971 without any change: the move was noise.

```
scores = [run(config, seed=s) for s in range(3)]  
print(f"{np.mean(scores):.3f} +- {np.std(scores):.3f}")    # 0.973 +- 0.002
```

- ▶ Three seeds before believing a change; adopt it when its mean beats the old mean by more than the spread (the Tuning Playbook: run the best trial several times to characterise the run-to-run variance).
- ▶ The seed is cheap on a table (1.7 s a run) and expensive on images. Spend it on the final candidate at least: the five-seed ensemble was the best number of the whole deck.

The situation

The task scores balanced accuracy; the loop prints accuracy. The majority model scores accuracy 0.700 and balanced accuracy 0.200. The task scores MAE; the loop trains and reports MSE (Day 16). The task scores ROC-AUC; the file holds 0/1 labels (Day 13).

```
from sklearn.metrics import balanced_accuracy_score # the judge's function
val_bal = balanced_accuracy_score(y_val, logits.argmax(1).numpy())
if val_bal > best:
    best, best_state = val_bal, copy.deepcopy(model.state_dict())
```

- ▶ The loss is for the optimizer; the metric is for you. Choose the checkpoint on the metric: here the validation loss is lowest at epoch 46 while the metric's plateau starts at 28.
- ▶ Import the metric from scikit-learn and compute it exactly as the task text says (average, threshold, which column). A metric written by hand at midnight is the usual off-by-one.

The situation

The epoch is chosen on the validation rows, then the validation score of that epoch is reported as the expected test score. Measured: choose the epoch on one half of the validation beats and score the other half at that epoch. Over five seeds the chosen half says 0.979, the other half says 0.968: 0.011 of optimism from choosing on the rows you score.

- ▶ The hidden test score lands near the *unchosen* number. Expect about a point less than the validation score that picked the checkpoint, more when the validation set is small.
- ▶ The public leaderboard is the same trap with a smaller set: the JOAI 2025 write-up that trusted its own folds rose about 30 places on the private half (Day 13). Never pick an epoch or a seed by the leaderboard.
- ▶ When the clock allows, choose with folds (Day 13): five out-of-fold numbers and their spread say what the test will say.

The situation

IOAI 2024, on site: “Your solution for each task should not exceed 5 minutes” of Colab CPU.
Polish 2025, noisy labels: the evaluation “may not take longer than 5 minutes with a GPU”. A notebook that is still training when the clock stops scores zero, however good its curve looked.

```
def predict_and_write(model, test_loader, ids, path="submission.csv"):
    model.eval(); preds = []
    with torch.no_grad():
        for xb, _ in test_loader:                # no shuffle: order kept
            preds.append(model(xb).argmax(1))
        sub = pd.DataFrame({"id": ids, "label": torch.cat(preds).numpy()})
        sub.to_csv(path, index=False)
    predict_and_write(model, test_loader, ids)    # after every improvement
```

- ▶ The first file goes out within fifteen minutes (the forest, or the majority class). Every improvement overwrites it only after the file is checked: rows, ids, class rate.
- ▶ Time one epoch before deciding the epochs, the seeds and the folds; a run you cannot repeat three times before the deadline is a run you should not start.

Training Runs: **Tips**

Tip: Budget the Clock Before the First Epoch

Share	What happens	Done when
first 10%	read the metric and the limits; majority and a Week 2 model; the data side; the first file is uploaded	a valid submission exists
next 40%	the recipe at its defaults, the dashboard, the rate sweep, early stopping	the network beats the Week 2 model, or does not
next 30%	one change at a time against the seed spread: capacity, dropout, weights, the loss	the ladder stops climbing
last 20%	seeds and their ensemble, the submission routine, the file checked, a last good copy	the file is uploaded with time to spare

- ▶ Time one epoch first and count: on the ECG beats a recipe run is 1.7 s and five seeds are 9 s, so the whole deck fits in a coffee break. On images an epoch is a minute, and the same budget allows four or five runs in total. Decide which four before starting.
- ▶ The shares move with the clock, the order does not: a file in the first tenth, seeds in the last fifth.

Tip: Three Functions and One Config

```
cfg = dict(lr=1e-3, wd=0.01, width=128, p=0.3, batch=64,  
           epochs=100, patience=10, seed=0)  
def fit(model, opt, loss_fn, loaders, cfg): # history, best state  
def evaluate(model, X, y): # eval(), no_grad(), the judge's metric  
def predict_and_write(state, test_loader, ids): # -> submission.csv
```

- ▶ Every run of the day is fit with a different cfg; every number of the day comes from evaluate; every file from predict_and_write. Three places to fix a bug instead of fifteen copies of a loop.
- ▶ The config is printed with the result, so a run can be repeated by reading one line. Day 17's lab built this shape (fit, evaluate, report); copy it into the first cell of Friday's notebook.
- ▶ A change is one key of the dictionary. If a change needs a new function, it is not one change.

Tip: Log Every Run in a Table

```
runs = []
runs.append(dict(name="plain SGD", best=0.944, epoch=51,
                 train_loss=0.0096, val_loss=0.152, secs=1.1))
runs.append(dict(name="recipe", best=0.972, epoch=28,
                 train_loss=0.082, val_loss=0.103, secs=1.7))
pd.DataFrame(runs).sort_values("best", ascending=False)
```

- ▶ The table is the ladder of Day 17's lab: plain, then AdamW, then weights, dropout, BatchNorm, early stopping, one row each. Reading it decides the next change; remembering it does not.
- ▶ Columns that earn their place: the metric at the best epoch, that epoch, the two losses at the end (the gap), the seconds. Add the seed and the mean over seeds when there are several.
- ▶ Write it to disk (`to_csv`) after every row; a runtime restart on Colab loses everything else.

Tip: Seed Everything, and Know What a Seed Does Not Do

```
def seed_all(s):  
    random.seed(s); np.random.seed(s); torch.manual_seed(s)  
    torch.cuda.manual_seed_all(s)                # the GPU too  
g = torch.Generator().manual_seed(0)            # the loader's order  
loader = DataLoader(ds, batch_size=64, shuffle=True, generator=g)
```

- ▶ Seeded, a run can be repeated: the same dials, the same batches, the same number, so a difference between two runs is the change you made. Unseeded, the difference is partly the dice.
- ▶ A seed does not make the noise small (the five-seed spread stays 0.004), and a GPU may still differ in the last digits between machines (PyTorch's reproducibility notes). The same seed on the grader's machine is therefore not a guarantee of the same score.
- ▶ Set it once at the top of the notebook and again before every `make_model()`, so a model built after a sweep starts from the same dials as one built before it.

Tip: Save Predictions, Not Only Models

```
np.save(f"val_probs_seed{seed}.npy", val_probs)      # [1500, 5] softmax
np.save(f"test_probs_seed{seed}.npy", test_probs)    # [n_test, 5], in order
P = np.mean([np.load(f"test_probs_seed{s}.npy")
              for s in range(5)], axis=0)             # the ensemble, free
```

- Probabilities on disk make every later decision free: the ensemble of five seeds, a threshold chosen on the validation probabilities (Day 13), a blend with the forest's probabilities, a re-written file after a format mistake.
- A saved model needs its code and its scaling to be useful again; a saved prediction needs nothing. Save both, and the predictions first.
- One file per run, named by the config and the seed; the run table of the previous tip points at them.

Training Runs: **Common Practices**

Practice: the Recipe Card, by Model Family

Dial	tables and signals (this week)	CNN-based vision (Week 4)	Transformer-based vision (Week 5)	language (Week 6)
start from	inputs-128-64-outputs, ReLU, last layer linear	a small pretrained CNN; from scratch only on tiny images	a small pretrained ViT, fine-tuned	a small pretrained BERT-style encoder, fine-tuned
optimizer, rate	AdamW, 10^{-3}	AdamW (or SGD with momentum), 10^{-3} to 10^{-4}	AdamW, 10^{-5} to 10^{-6}	AdamW, 10^{-5} to 10^{-6}
batch size	64 (32 to 256)	4 to 32, set by memory	4 to 32, set by memory	1 to 16 sequences
epochs, schedule	up to 100 with patience 10 to 30; no schedule, or on a plateau	5 to 300, more with heavy augmentation; cosine decay per epoch	5 to 50; cosine decay with a warm-up of 3 to 5% of the steps	1 to 10 (large language models 1 to 3); cosine or linear decay per step, warm-up 3 to 5%
regularisation	dropout 0.2 to 0.5, weight decay 0.01, class weights	augmentation first (flips, crops, colour, MixUp, CutMix), then weight decay	augmentation, weight decay 0.05, dropout 0.1	dropout 0.1 inside the model, weight decay 0.01, early stopping
inputs, normalisation	standardised columns; BatchNorm inside	224×224 with the pretrained mean and spread; BatchNorm inside	224×224 with the pretrained mean and spread; LayerNorm inside	token ids, length 128 to 512 (prototype at 256); LayerNorm inside

- Keep the batch size fixed and tune the rate; if memory forces a new batch size, scale the rate with it (linearly for SGD, by the square root for AdamW). Early stopping first to find the epoch range, then fixed epochs with a cosine schedule over them.
- Tuning order on every family: rate and epochs, then the schedule, then augmentation added gradually, then a bigger model or input; start tiny and scale once the pipeline is stable. The first column is measured in this deck; the others are the defaults of Weeks 4 to 6.

Practice: One Change at a Time, Against the Spread

Run	best	at epoch	what changed
plain network, SGD 0.05	0.944	51	the baseline network
the recipe, one seed	0.972	28	AdamW, class weights, dropout, BatchNorm, early stopping
the recipe, five seeds	0.971 to 0.975	25 to 37	nothing: the noise
the five averaged	0.979		the ensemble

- ▶ The jump from 0.944 to 0.972 is five changes at once. Day 17's lab took them one by one, same seed, and found most of it came from AdamW and the weights, with dropout and BatchNorm steadying the curve: that is the only way to know which dial to keep when time runs short.
- ▶ The Tuning Playbook's rule in plain words: decide which one thing you are testing, hold everything else fixed, and compare its best score with the old one *and* the spread. A move inside the spread is kept only if it is cheaper.
- ▶ Never tune the epoch count: fix a generous maximum, let patience stop, and read the kept epoch instead.

Practice: What Wins Points, What Loses Them

Wins points	Loses points
a Week 2 model submitted in the first tenth of the clock	the first file written in the last ten minutes
the loss checked at initialisation, one batch over-fitted	an hour tuning a pipeline that cannot fit 64 rows
the rate swept by powers of ten	one rate, chosen by habit
the checkpoint kept by the judge's metric, patience to stop	the last epoch, or the epoch of the lowest loss
<code>eval()</code> and <code>no_grad()</code> in evaluate	dropout and BatchNorm scoring the validation set
class weights when the metric is balanced	accuracy watched while balanced accuracy is scored
three seeds to compare, five to ensemble	a 0.003 "improvement" adopted
the out-of-fold loss when labels look noisy	training on every row because the file said so
the file checked: rows, ids, order, class rate	a shuffled test loader

```
final = make_recipe(); final.load_state_dict(torch.load("best.pt"))
final.eval(); probs, ids = [], []
test_loader = DataLoader(test_ds, batch_size=512)    # no shuffle, no drop_last
with torch.no_grad():
    for xb, idb in test_loader:                      # yields (x, id)
        probs.append(torch.softmax(final(xb), 1)); ids += list(idb)
P = torch.cat(probs).numpy()
sub = pd.DataFrame({"id": ids, "label": P.argmax(1)})
sub.to_csv("submission.csv", index=False)
chk = pd.read_csv("submission.csv")
print(chk.shape, chk["label"].value_counts(normalize=True))
```

- ▶ Best state, eval(), no_grad(), batches in the test order, the id from the Dataset (Day 19), the file read back: the shape must match the sample submission, the ids its ids, the class rate the training rate within reason.
- ▶ Probabilities when the metric wants them (ROC-AUC, log loss), labels when it wants labels, the regression number as it is: the pairing table of Day 16 decides the last line.

Training Runs: **Olympiad Practice**

Task (adapted)

The ECG beats of Days 16 and 17: 2,000 training beats of 150 voltages, five classes, 70% normal, a validation set of 1,500 beats with clean labels, a hidden test set scored by balanced accuracy. This time 15% of the *training* labels are wrong, nobody knows which, and the clock gives two hours on a laptop CPU. Plan the two hours.

- ▶ **Question 1:** what is uploaded within the first fifteen minutes, and roughly what does it score?
- ▶ **Question 2:** which numbers on the dashboard reveal that the training labels are noisy, and which move of this deck answers them?
- ▶ **Question 3:** how is the final model chosen between the recipe on all rows and the recipe on the cleaned rows, how many seeds does it get, and what number do you expect on the hidden test?

Adapted from II Olimpiada AI (Poland, 2025), stage 1: the ECG data of task 3 and the noisy-label setting of task 4; numbers from `scripts/day20_practice_figures.py`.

1. **First fifteen minutes:** standardise with the training rows, `check(ds)`, then the forest with `class_weight="balanced"` on the noisy labels (1 s) and its file. On clean labels it scored 0.931; with 15% noise expect less, and it is still a file. The majority baseline is 0.200: the floor the file must clear.
2. **The dashboard tells:** the training loss of the recipe stops far above zero (a network cannot fit 314 rows whose label contradicts their neighbours) while the validation metric plateaus well below the clean 0.972: 0.888. The out-of-fold loss histogram is bimodal; the 15% highest-loss rows are 74% wrong labels. Drop them and retrain: 0.912. Cost: 5 folds of 12 epochs plus one run, about 10 s here.
3. **The final model:** compare the two candidates with three seeds each; cleaned rows win by two points, well outside the spread of 0.004. Give the winner five seeds and average the softmax (the ensemble added 0.007 on clean labels). Expect the hidden score about a point below the validation number that chose the epoch (the optimism measured at 0.011): roughly 0.90. Upload with time left; the run table and the probabilities on disk are the record.

Training Runs: **Check Yourself**

Answer before turning the page.

1. **Definition.** Name the five numbers of the dashboard. A fresh five-class network reports a training loss of 1.683 at initialisation and 0.000 after one epoch: which of the two is the alarm, and why?
2. **Mechanism.** A run prints: training loss 0.004, validation loss 0.52 and rising since epoch 14, validation balanced accuracy best 0.91 at epoch 14, now 0.89 at epoch 60. What do you submit, what do you change, and why is the rising loss not by itself the problem?
3. **Code.** Three mistakes in this loop cost points without raising an error. Name them and the fix.

```
X_val = (X_val - X_val.mean(0)) / X_val.std(0)
for epoch in range(60):
    model.train()
    for xb, yb in train_loader:
        loss = loss_fn(model(xb), yb)
        opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        val_bal = balanced_accuracy_score(y_val, model(X_val).argmax(1))
pred = model(X_test).argmax(1)
```

1. Training loss, validation loss, the judge's metric, the best score with its epoch, the seconds per epoch. 1.683 is fine: a guessing five-class network sits near $\ln 5 = 1.609$. A loss of 0.000 after one epoch is the alarm: nothing real is learnt that fast; the labels are in the inputs, or the validation rows are in the training set.
2. Submit the dials of epoch 14 (the kept `state_dict`), never epoch 60. Next change: regularise and stop earlier (dropout, weight decay, patience 10), then a smaller or better-fed model if the gap stays. The rising validation loss with a flat metric means the wrong answers are getting confident (Day 16); the metric is what chooses the checkpoint, and here it did fall too (0.91 to 0.89), which is the real overfitting signal.
3. (a) The validation rows are standardised with their own mean and spread: use the training mean and spread for every split (a leak, and a different scale from the test rows). (b) `model.eval()` is never called: dropout and BatchNorm score the validation set in training mode (0.934 instead of 0.971 in this deck). (c) The last epoch predicts the test set: keep `best_state` by `val_bal` and load it, with `eval()` and `no_grad()`, before predicting.

Training Runs: **Limitations & What Comes Next**

- ▶ **One table, one laptop.** Every number here is from 2,000 beats of 150 numbers, where a run costs seconds. The moves are the same on images and text; the budget that decides how many of them you can afford is not.
- ▶ **The recipe card is a default, not a law.** It is what people start from; the sweep, the dashboard and the spread decide when to leave it, and a Week 2 forest sometimes ends the discussion.
- ▶ **The noisy-label trick assumes random noise.** Labels wrong in a pattern (every hard case marked normal) look like hard rows, and dropping high-loss rows then throws away the cases that matter. Look at the dropped rows before training without them.
- ▶ **Validation numbers are estimates.** The optimism of 0.011 was measured on 750 rows; a smaller validation set is noisier, and the leaderboard is smaller still.

The common mistake

Changing three dials at once, reading the result as the effect of one, and keeping all three. One change, one seed spread, one row in the table.

- ▶ **Week 4:** images and convolutions. The routine, the dashboard and the decision table are unchanged; the Dataset gains augmentations, the epochs get expensive, and the budget tip becomes the first thing you do.
- ▶ **Week 5:** pretrained models and fine-tuning: the same loop with a different starting point, where the rate sweep and early stopping matter even more because every epoch is slow.
- ▶ **Friday:** the deck's routine frame and the two tables are the ones to keep open; the practice task is the rehearsal.

Training Runs: **Summary**

- ▶ **The routine:** metric and clock, a Week 2 model and its file, the data side, the recipe at its defaults, the dashboard, one change at a time, seeds, the submission routine.
- ▶ **The dashboard:** two losses, the judge's metric, the best epoch, the seconds; $\ln k$ at initialisation; the decision table names the failure from the shape.
- ▶ **Tricks:** the rate by powers of ten; the kept checkpoint with patience; class weights for balanced metrics; five seeds averaged (0.979); the out-of-fold loss on noisy labels (0.888 to 0.912); the loss at init and one batch to zero; the forest first.
- ▶ **Mistakes:** `eval()` forgotten (0.934 for 0.971); the last epoch; unscaled inputs (0.659 for 0.944); one seed; the wrong number watched; an optimistic validation score (0.011); no file on the clock.
- ▶ **Practices:** the recipe card, one change against the spread, what wins and loses points, and a submission routine that reads the file back.

Training Runs: **References & Resources**

- [1] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, chapter 11: practical methodology (baseline, instrument, the learning rate first, one change at a time).
<https://www.deeplearningbook.org>
- [2] Karpathy, A. (2019). A recipe for training neural networks.
<https://karpathy.github.io/2019/04/25/recipe/>. The loss at init, overfit one batch, “Adam is safe”, ensembles.
- [3] Godbole, V., Dahl, G., Gilmer, J., Varun, C. and Nado, Z. (2023). *Deep Learning Tuning Playbook*.
https://github.com/google-research/tuning_playbook. Retrospective checkpoint selection, batch size, run-to-run variance, one scientific hyperparameter per experiment.
- [4] Prechelt, L. (1998). Early stopping, but when? In *Neural Networks: Tricks of the Trade*, Springer.
- [5] Smith, L. (2018). A disciplined approach to neural network hyper-parameters: learning rate, batch size, momentum, and weight decay. arXiv:1803.09820. Reading the validation loss early in training.
- [6] Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. *ICLR*. AdamW.
- [7] Srivastava, N. et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. *JMLR* 15; Ioffe, S. and Szegedy, C. (2015). Batch normalization. *ICML*.

- [8] PyTorch documentation, Reproducibility. <https://pytorch.org/docs/stable/notes/randomness.html>
- [9] II Olimpiada AI (Poland, 2025), stage 1: task 3 (ECG beats, the data of this deck) and task 4 (noisy labels, selection by per-sample losses, a fixed network, five minutes on a GPU).
<https://github.com/OlimpiadaAI/II-OlimpiadaAI>
- [10] IOAI 2024, on-site round, *Lost in Hyperspace*: five minutes of CPU per task, linear regression on engineered features. <https://github.com/IOAI-official/IOAI-2024>
- [11] NEOAI 2025, *Tricky Table*: gradient boosting as the whole solution; JOAI 2025 solutions: stratified folds, boosting on sensor columns, ensembles. <https://ioai-japan.org/joai2025-solution/>
- [12] Days 13, 16 and 17 of this course: the evaluation routine; the loop, the losses and the curve; the dials and their measured gains.
- [13] All plots and every number quoted: `slides-builder/scripts/day20_practice_figures.py` (baselines, the rate sweep, the dashboard, the `eval()` mistake, seeds and their ensemble, class weights, noisy labels, the optimism of epoch selection, unscaled inputs).